PROJECT - 2
Distributed Parallel File System

Shashank Appana | sba6069@psu.edu Prudhvi Gadupudi | gvp5349@psu.edu

Architecture Design:-

We are storing files as blocks in the file server. We will uniquely identify each block and store it in the respective file server.

For example:-

Pfs_file218042893831734055175_server_1_342528_343039 is a block in which
Pfs_file218042893831734055175 is a unique file identifier
server_1 indicates which server that block is in
342528_343039 indicates the start offset and end offset of the bytes that are being store in this block.

Metadata in Metadata server:-

We store metadata in a map<filename, PfsMetadata*> structure. This structure acts as the directory for our files in our PFS. PfsMetadata* points to the node containing the metadata.

```
message PfsMetadata {
        .
        .
        .
        string unique_file_identifier = 7;
        repeated FileServerBlock block_addresses = 8;
}
```

Along with the given metadata information as per the requirements, we are also storing the unique file identifier, which uniquely identifies the file through its lifetime. Block addresses will have the final block names and related data.

During creation of file:-

We create a unique identifier for the file and construct metadata and we store the pointer to that metadata in the directory map explained above.

During opening of file:-

We assign a unique file_descriptor_id and construct FileDescriptor which contains pointer to metadata, access mode etc. and store it in the openFiles directory

During the deletion of file:-

We delete the mapping in the filename to the metadata directory. If there are any open files we will delay the deletion of the file and set marked for the deletion as true in the metadata.

During the closing of the file:-

We will delete the entry from the open files directory, if the metadata has marked for delete set, we will delete the file.

Token Management:-

Our token management works based on the invariant that metadataserver will process only one token request at a time. That means metadataserver **at anytime will have the latest tokens with it.** And reads/ writes operations will eventually conclude and will respond to token revocation requests.

Step 1:- Clients if they don't have the required tokens, they will request the token and wait for the response. For waiting we created an acknowledgment map, where each entry will have a conditional variable to wait on. Its key is a unique value, both client and server will exchange that unique value in its response. So now we have a request sent from the client to metadataserver and client is waiting on the conditional variable.

Step 2:-

Server on receiving the request from the client, will read the stream and creates a thread to process the request. This newly created thread will wait until it gets the mutex lock, by which we can ensure that the server will only process one token request for each file at a time. Once it gets the mutex, it adds all the missing tokens to that client's entry in the data structure and sends a revocation request to all clients which have those tokens. On receiving of acknowledgment for all revocation requests(we identify the revocations acknowledgments in the same way using unique_id, conditional variable mapping) we will send back acknowledgment to the token request. In the acknowledgment we also send new tokens to the client.

Step 3:-

After receiving the acknowledgment, the stream thread that reads the acknowledgment will signal the conditional variable and the thread continues with its operation of reading or writing. **Before entering the reading or writing it will add this range to the busy range.** When we receive a revocation request if the range is in busy range, we will wait till the range is marked not busy. For synchronising this we have a conditional variable on entry of the busy range array.

Cache Management:-

Cache is managed at block, when we are writing/reading a block to file server we will have a cache in between. For LRU cache implementation we have use doubly linked list and a map. Map stores direct mapping between block name and Cache node. While doubly linked list is used to manage Least recently used element in the cache. Whenever we are reading or writing from the cache we will move that block to the head of the list. And we will handle capacity misses by removing them from the tail.

Cache Invalidations and Write backs:-

On new tokens:- We might have cached a block because of the different offset but for the new tokens we never had the cache, so we can't guarantee that the entries in our cache for the new tokens is correct content, so we will invalidate the cache and write back.

On File Close:- On file close we will invalidate the cache and write back.

On Token Revocation:- when metadata asked us to revocate our tokens we will invalidate and writeback all dirty blocks.

On Capacity Miss:- When max cache capacity is reached, we will write back the dirty block and delete this from the cache.

Individual Contribution:-

Shashank:- grpc connection establishment and api methods without tokens and cache, Cache Design and Implmentation, Token Design, Testing & Debugging.

Prudhvi:- grpc connection establishment and api methods without tokens and cache, Token Management, Testing & Debugging.